CX Sprint 2026.05.07

Home Portal (1) | SmartApp (2) | Servicing (2) | Admin Portal (1) | Sitewide (1) | Tech Enhancem

Home Portal (1 items)

🆕 NEW

Include Assigned Borrower in Upload payload

#103662

Summary:

- Document Uploads Now Correctly Identify the Associated Borrower
- What changed: Enhanced the Home Portal document upload payload to include borrower identification when a task is assigned to a specific borrower or borrower pair.
- Who benefits: Borrowers uploading documents through Home Portal - their files are now correctly associated with the right applicant in ClearDocs, reducing manual processing for loan teams.
- Technical context: When a task is assigned to a single borrower, that borrower is passed in the POST to ClearDocs; when assigned to a pair, both borrower identifiers are included in the payload.

SmartApp (2 items)

✨ ENHANCED

Update Okta template for Partner Portal support

#103966

Summary:

- Extended Okta Login Template to Support Partner Portal
- What changed: Updated the Okta hosted sign-in template to support Partner Portal authentication and aligned its visual design with the partner portal's white-header login page.
- Who benefits: Mortgage brokers and TPO partners logging into the Partner Portal - they now see a consistent, brand-appropriate login experience using the Okta hosted login flow.
- Technical context: Extended src/static/okta-templates/login.html; design aligned with the Partner Portal white-header login redesign per GHE issue #63 in pilot-program/partner-portal.

✨ ENHANCED

Disclaimer for NextMortgage should be updated

#104363

Summary:

- Updated Legal Disclaimer Text for NextMortgage
- What changed: Updated the SmartApp short and long disclaimer text for the NextMortgage brand to reflect current licensing and regulatory information, including state-specific license numbers.
- Who benefits: Borrowers applying through the NextMortgage brand - the disclosure text they see now accurately reflects NextMortgage's current regulatory status and NMLS ID 1937279.
- Technical context: Updated disclaimer covers California (DFPI CRMLA #60DBO-117725), Colorado, Texas, Georgia, Ohio, Oregon, and additional state-specific licensing language.

Servicing (2 items)

✨ ENHANCED

Paid in Full account Loan Details card updates_part 2

#103667

Summary:

- Completed Updates to the Loan Details Card for Paid-in-Full Accounts
- What changed: Completed the second phase of Loan Details card updates on the Mortgage Account dashboard for LoanServ accounts that have been fully paid off.
- Who benefits: Homeowners who have paid off their mortgage - the account dashboard now clearly reflects their paid-in-full status with accurate balances and appropriately hidden payment options.
- Technical context: Builds on #90905 deployed last sprint; Part 2 completes the remaining acceptance criteria for paid-in-full account display in LoanServ-backed accounts.

✨ ENHANCED

Update Payment Amount and Due to Active Bankruptcy

#103790

Summary:

- Updated Payment Display for Borrowers in Active Bankruptcy
- What changed: Updated the Home Portal Mortgage Account dashboard to show the post-petition payment amount and due date for accounts with an active bankruptcy status.
- Who benefits: Homeowners in active bankruptcy - the payment amount and due date shown on their dashboard now accurately reflects their bankruptcy repayment terms rather than the standard mortgage payment.
- Technical context: Uses the bankruptcy detection logic from #101485 to conditionally swap displayed payment data to post-petition values; the change only applies when the BK flag is 'A' (Active).

Admin Portal (1 items)

🆕 NEW

Brand Specific Emails | Next Mortgage

#103788

Summary:

- Added Next Mortgage Brand to Admin Portal Email Trigger
- What changed: Added Next Mortgage as a selectable organization in the Admin Portal brand dropdown when triggering emails such as account unlock notifications.
- Who benefits: Admin users managing Next Mortgage accounts - they can now trigger brand-accurate emails under the Next Mortgage identity rather than defaulting to another brand.
- Technical context: Extends the existing Admin Portal email trigger that previously supported only BankCMG and CMG Home Loans; Next Mortgage email routing follows the same brand-selection pattern.

Sitewide (1 items)

🆕 NEW

Implement Email Branding with Email module - CMG and BankCMG

#103668

Summary:

- Implemented Brand-Specific Email Templates for CMG Home Loans and BankCMG
- What changed: Wired up the Email module so that transactional emails sent from SmartApp and Home Portal reflect the originating brand (CMG Home Loans or BankCMG) rather than a generic template.
- Who benefits: Borrowers and loan officers receiving email notifications - emails now display the correct branding, logos, and sender identity matching the brand that originated the transaction.
- Technical context: Brand selection uses the Email module routing logic; CMG and BankCMG configurations tested across all notification types from both SmartApp and Home Portal entry points.

Tech Enhancements (14 items)

TECH

Remove oktaSsoToken post/get logic (no longer needed with hosted Okta login)

#99167

Summary:

- Removed Obsolete Okta SSO Token Relay from SmartApp
- What changed: Removed the client-side oktaSsoToken POST/GET endpoints and supporting database infrastructure used for cross-domain SSO redirects during authentication.
- Who benefits: Engineers maintaining the codebase - fewer dead-code paths and database endpoints reduce maintenance overhead and potential attack surface.
- Technical context: This pattern stored Okta tokens temporarily in the database during cross-domain redirects; the hosted Okta login page now handles token exchange natively, making this relay unnecessary.

TECH

Add [DisableGlobalAuthorize] attribute to ExternalController in ConX API

#100268

Summary:

- Fixed Authorization Conflict on External API Controller
- What changed: Added the [DisableGlobalAuthorize] attribute to ExternalController in ConX API to prevent the Plinth global-authorize filter from conflicting with its per-method API-key authorization.
- Who benefits: Engineering and ops teams - eliminates spurious authorization errors in App Insights caused by the duplicate authorization check.
- Technical context: ExternalController uses explicit [ApiKey] attributes on every method; the global Plinth filter was incorrectly intercepting these calls before method-level auth could evaluate.

TECH

Update Vulnerable Packages in ConX API

#101092

Summary:

- Updated Vulnerable Dependencies in ConX API
- What changed: Upgraded AutoMapper, Scriban, and MimeKit to resolve moderate and high-severity vulnerabilities flagged in the ConX API codebase.
- Who benefits: All users of the platform - running production code without known high vulnerabilities reduces security risk exposure.
- Technical context: Unresolved vulnerabilities were also blocking Entity Framework migration work; this unblocks continued EF development alongside the security remediation.

TECH

Update Vulnerable Packages in ConX Admin

#101094

Summary:

- Updated Vulnerable Dependencies in ConX Admin
- What changed: Upgraded the Scriban package in ConX Admin to resolve moderate and high-severity vulnerabilities in the admin codebase.
- Who benefits: All users of the admin platform - removes known security vulnerabilities from production-deployed code.
- Technical context: Aligned with the parallel ConX API package update (#101092); both addressed in the same sprint to maintain parity between services.

TECH

SmartApp Save Flow ⬩ Race Conditions, Missing BorrowerGUID, and Partial Failure Handling

#102732

Summary:

- Fixed Race Conditions and Data Loss in SmartApp Save Flow
- What changed: Refactored the SmartApp wizard save flow to resolve race conditions that caused NULL BorrowerGUID errors, duplicate entity creation, and silently discarded save results.
- Who benefits: Borrowers filling out loan applications - eliminates a class of data-loss bugs where employment, income, liability, and real estate entries were silently dropped or duplicated during multi-borrower saves.
- Technical context: The prior forkJoin parallel-save pattern fired child entity endpoints before the parent borrower POST returned a GUID; 411 SqlExceptions over 30 days confirmed active production data loss (usp_InsertEmployment: 283, usp_InsertIncome: 70, usp_InsertLiability: 30, usp_InsertRealEstateOwned: 28).

TECH

SmartApp Submit Skips Save ⬩ Dirty Data Lost on Loan Application Submission

#102733

Summary:

- Fixed Lost Form Data When Submitting a Loan Application
- What changed: Updated the SmartApp submission flow to save any unsaved form changes before dispatching the application, preventing silent data loss on the prequalification review page.
- Who benefits: Borrowers - any last-minute edits made on the review page before clicking 'Submit Application' are now preserved and included in the submitted application.
- Technical context: The wizardComplete action in wizard.store.ts skipped calling save() before setting complete: true; the fix ensures save is called in smart-1003.component.ts prior to api.submit(guid).

TECH

XSS Pipe Remediation ⬩ Replace safeHtml with Whitelist Sanitizer

#102817

Summary:

- Replaced Unsafe HTML Pipe with Whitelist-Based Sanitizer
- What changed: Replaced the safeHtml Angular pipe - which bypassed all XSS sanitization - with a new sanitizedHtml pipe that enforces a whitelist of safe tags and attributes.
- Who benefits: All borrowers and staff using the platform - eliminates a class of cross-site scripting vulnerabilities across 12 template locations.
- Technical context: The new sanitizedHtml pipe uses DOMParser to strip non-whitelisted tags and attributes, then passes output through Angular's SecurityContext.HTML sanitizer as a second pass; allowed tags are limited to b, i, strong, em, a, br, p, ul, ol, li, and span.

TECH

innerHTML Surface Reduction ⬩ Convert Error Bindings to Text Interpolation

#102823

Summary:

- Reduced XSS Attack Surface by Converting Error Bindings to Safe Text Interpolation
- What changed: Converted approximately 10 [innerHTML] bindings that rendered plain error messages to Angular text interpolation, which auto-escapes HTML content.
- Who benefits: All platform users - eliminates unnecessary XSS vectors in error message display across loan selection, upload, borrower document, and task list components.
- Technical context: None of the converted bindings contained HTML formatting, making this a zero-risk refactor; affected components include loan-select, upload, borrower-docs-upload-auth, borrower-docs-upload-contact, documents-list-task, and task-list.

TECH

Match borrowers by identity after loan creation, not array position

#102843

Summary:

- Fixed Borrower Identity Mismatch After Initial Loan Creation
- What changed: Updated createLoanApplicationOnApi to match borrowers from the API response by identity (borrowerGUID) rather than array position when merging form data after a loan is first created.
- Who benefits: Borrowers applying as a pair - eliminates a class of bugs where co-borrower data could be silently applied to the primary borrower due to position-based array merging.
- Technical context: Ramda's mergeDeepRight replaces arrays wholesale and Angular's FormArray.patchValue patches by index; both break when loanFull returns borrowers in a different order than the form expects.

TECH

Harden loanFull borrower sort to handle missing isPrimary

#102844

Summary:

- Hardened Borrower Sort to Handle Missing Primary Flag
- What changed: Updated the loanFull borrower sort in api.stores.ts to include a fallback when no borrower has isPrimary set, preventing co-borrower data from appearing in the primary borrower position.
- Who benefits: Borrowers in joint loan applications - prevents a scenario where both applicants could be treated as co-borrowers, potentially mixing their loan data.
- Technical context: The prior filter-based sort treated null, undefined, and false identically; when isPrimary is absent, usp_SearchBorrowers returns borrowers ORDER BY DateInserted DESC, placing the co-borrower first and inverting expected order.

TECH

API ⧄ Ensure at least one borrower has IsPrimary after creation

#102846

Summary:

- Added Server-Side Guarantee That Every Loan Has a Primary Borrower
- What changed: Updated LoanApplicationManager.SetDefaults in ConX API to validate that at least one borrower has IsPrimary set after loan creation and to assign it when missing.
- Who benefits: Borrowers applying as a pair - prevents silent data errors downstream when neither borrower is designated as the primary applicant.
- Technical context: The prior null-coalescing assignment was skipped whenever the client submitted borrowers in the POST body; with IsPrimary typed as bool? (nullable), a null value from deserialization could pass validation unchallenged.

TECH

Fix positional borrower merge in formGroupCreateExisting

#102868

Summary:

- Fixed Borrower Data Mix-Up When Resuming an In-Progress Loan Application
- What changed: Replaced hardcoded array-index merging in formGroupCreateExisting with identity-based borrower matching, preventing primary and co-borrower data from being swapped when a user returns to a saved application.
- Who benefits: Borrowers resuming a joint loan application - their personal information now reliably appears in the correct section rather than being transposed with their co-borrower.
- Technical context: The prior merge assumed borrowers[0] is always primary and borrowers[1] is always co-borrower; if loanFull returned them in a different order when the isPrimary sort failed, data was silently cross-applied to the wrong borrower.

🔧 TECH

Replace hardcoded positional borrower access with identity-based lookups

#102869

Summary:

- Replaced Positional Borrower Array Access with Identity-Based Lookups
- What changed: Replaced all hardcoded borrowers[0]/borrowers[1] array index access across the smart-1003 module with lookups by isPrimary flag and borrowerGUID.
- Who benefits: Borrowers in joint loan applications - ensures VOIE, VOA provider selection, and co-borrower detection always operate on the correct applicant regardless of array order.
- Technical context: Affected locations include smart-1003.service.ts VOIE/VOA provider aggregation and review.component.ts isCoBorrower getter; this is the final step in a sprint-wide borrower identity hardening effort.

🔍 SPIKE

Improve Sensitive-Data Logging Infrastructure for Request Logging [16HRS]

#101112

Summary:

- Researched a Targeted Strategy for Redacting Sensitive Data from Request Logs
- What was researched: A more precise approach for redacting sensitive data from request logs, as an improvement over the existing method that disables logging entirely for sensitive API paths.
- Key findings: The current brute-force approach sacrifices observability; configurable field-level redaction was evaluated as the preferred replacement to preserve audit capability while protecting sensitive borrower data.
- Next steps: Findings unblock a proper implementation of field-level log redaction that balances security and operational visibility.

© 2026 Consumer Experience Team | Generated from Azure DevOps